

Internship Experience Review

So this AI internship I learned to work through essentials that is required to know and understand for deployment and working of full stack web applications, I learned strongly through trial and error because coding using AI requires precision in PRD and Prompt engineering, I was careless with the prompts at first thinking the AI will figure things out but I was wrong I had to be specific with almost everything and majorly for frontend and UI the themes, the designs everything required for it to execute the particulars I want, I had to go through different inspirations websites GitHub repos in order to get an understanding of what the kind of UI and frontend I want for this product, the technical team made it easier to give us the specific websites that we had to go through in order to find things we want. I also learnt about several other essentials in a working space, other technical skills required in SWE like git and GitHub, now that AI coding is booming what makes us unique in the job market like being able to still understand the code breakdown errors and architectural inaccuracies still makes a difference in the job market. I was also given an idea on several other domains that is discoverable and operable when studying the computer sciences.

I was assigned 2 projects to work on 1 is the major project, which is DizruptOS, an enterprise grade workforce management platform. Instead of a traditional dashboard, I implemented a full in browser desktop Operating System (MacOS). One of the major ups of my application is role based access control I made it strict and proper in order for dynamic viewing and access capabilities depending on the role of the person in the company, so each person depending on their role will have different access, different features and different viewing dynamic and tailored to their role. I had implemented a bunch of initial features at first, but it posed a problem deviating me from the initial problem statement given to me, it made the additional features a lot unnecessary. The major problem statement for this is the Management view of every employee how much they are loaded with tasks, if they are free easy assigning of tasks to them, easy reassigning of tasks from overloaded employees, and project and employee management system

in general. I tried to incorporate AI driven intelligence into this product for features like a copilot chatbot feature in the web application. The system will be able to analyse the project idea its tasks and will be able to provide suggestions on what can be done next in the project, will provide recommendations on the people who are overloaded which tasks from them can be assigned to whom, easy drag and drop kanban matrix that updates companywide metrics in real time. I personally learnt a lot about how to set up supabase which was new to me completely how do I incorporate and set it up to my project, I learn advanced react components, how AI coding can put security of the application at and edge, I learnt how to effectively and uniquely prompt AI to implement the exact things I have provided it to give it as much context that it requires for its proper execution, I learnt how to build a complete application by breaking it down to literally mini apps as the features present in the application that easily talks to and interacts with each other so the changes made by the higher authorities are live making the product easier to maintain and upgrade. I was able to get various inspirations from different websites recommended to me by the technical team and chose well for to build fluid, physics-based animations. I learned to make software that doesn't seem like AI slop or like a rigid dashboard instead I engineered good prompts and gave it proper inspirations so that it can make it like a premium, highly responsive Apple product like experience(literally). One of the major learnings I would say was I was able to learn on how already existing applications work (Linear, Monday.com) and so how to take inspirations from their UI how to make it work how to not overwhelm the user in first glance, a major problem I faced was that my seed data not connecting properly because initially my seed data was randomized I put it in just to showcase it I didn't intend to connect the data, that was a major problem that the technical team faced while trying to work through my application, I fixed that provided new easily understandable and easily connectable data changed the seed.sql file completely and updated it in the supabase after which I pruned and hid the additional unnecessary features, performed error handling and testing prompts on multiple iterations and finalized the application completely.

Technical Stack

- Core Framework: Next.js 14 (App Router), React 18, TypeScript
- State Management: Zustand (for global, cross-window state)
- Styling & Animation: Tailwind CSS, Framer Motion, Radix UI Primitives
- Backend & Database: Supabase, PostgreSQL, Edge Functions
- Drag & Drop Engine: @hello-pangea/dnd
- Architecture Style: Web-based Desktop Environment / Micro-frontend Simulation

The above is the technical stack I used, although I might not be proficient in coding with all of them. It would at least be easy for me to work with these tech stacks for future projects.

The second project I worked on is another web application a Talent Repository, it's a full stack Talent Management and applicant tracking system (ATS). Designed with a premium dark mode aesthetic, it serves as a repository for HR teams to source, evaluate and track candidates across a complex pipeline.

Tech Stack:

- Frontend: React (Vite), TypeScript, HTML/Vanilla CSS (Custom Design System)
- Backend & Database: Supabase (PostgreSQL, GoTrue Auth)
- State Management: Zustand
- Animations: Framer Motion
- Parsing Engines: pdf.js (Resume Parsing), read-excel-file (Bulk Ingestion)
- Application Link: <https://talent-intelligence-hub-web.vercel.app>

The above is the tech stack I used to build this application. In this application I built a bulk ingestion engine taking inspiration from github repos of preexisting parsers, it will be able to ingest complete csv or excel sheets jumbled up spreadsheets. Instead of failing when column

headers don't match, the engine uses pattern recognition (regex) and fuzzy-matching heuristics to infer names, compensation, skills, and contact info straight from the raw cell data. To eliminate manual entry of data while addition of talents happen in the form of resume, I built an AI resume parser to eliminate manual data entry, also got inspirations from preexisting functionality and open source repos, the application extracts text from uploaded PDFs and uses advanced heuristic mapping to automatically populate a candidate's profile (Name, LinkedIn, Job Title, Skills, and Experience) all without making expensive API calls to a backend server. I implemented a Kanban Board and tabular Talent Directory. Using Zustand, state is synchronized globally so that moving a candidate from "New" to "Shortlisted" on the Kanban board instantly updates the metrics on the Dashboard, the Directory, and the Comparison table. I designed a relational Postgres schema hosted on Supabase. I told AI to write RLS scripts so that users can see candidate data belonging to their specific organization. I learned that you don't always need a heavy backend server. By utilizing the user's own browser to parse heavy PDF files and large Excel spreadsheets, I drastically reduced server costs and network latency, resulting in a much faster, offline-capable experience. I ran into a problem post deployment which was pretty silly whenever I switch tabs and come back to the same application again I face problems of loading screen flashing and the updates I have made not being saved properly which turns out that my supabase configuration URL had a particular commit in it so any updates I made after was not reflected so I dealt with the problem and fixed it. I ran multiple tests and error handling prompts through AI to ensure stability of the application. Writing supabase RLS taught me to enforce security even in the database layer. I also looked through a couple of websites to get inspirations and animated experience of the login page UI which made me understand how specific I had to be for AI to code the UI, I had to go through multiple iterations to get the animation proper and similar to what I wanted in the UI. The biggest lesson was realizing that users will rarely format data the way developers expect. Building the CSV bulk importer taught me how to write defensive algorithms that anticipate missing headers, mismatched columns, and edge cases, rather than just throwing error messages at the user. So it was a great

experience of failing and then getting back up in cycle till the very end as it should be.